

Erika

Erika is a lightweight voice assistant designed for a Raspberry Pi 5 running Raspberry Pi OS Trixie with Wayland/labwc.

The assistant continuously listens for the wake phrase:

```
hey jamal
```

After detecting the wake phrase, Erika records and transcribes the following command, executes supported local actions, and falls back to Gemini for general questions.

Erika also provides spoken feedback through a USB speaker.

Features

Erika currently supports:

- Wake-word detection using Vosk
- Continuous microphone input using ALSA
- Speech-to-text command recognition
- Interruptible commands
- Spoken feedback using eSpeak NG
- Gemini queries through the Ulauncher Gemini Direct extension
- YouTube music playback
- Application control
- Automatic startup through a systemd user service

A new wake phrase can interrupt an action that is currently running.

For example:

```
hey jamal play erika
```

While that command is running, you can say:

```
hey jamal what time is it
```

and Erika will interrupt the previous action and process the new command.

Hardware

The current hardware configuration is:

Raspberry Pi

- Raspberry Pi 5
- Raspberry Pi OS Trixie 64-bit
- Wayland

- labwc compositor

Microphone

DJI Wireless Mic receiver

ALSA device:

```
plughw:CARD=Rx,DEV=0
```

Audio is converted to:

```
16-bit signed PCM  
16 kHz  
Mono
```

Speaker

KM_B2 Digital Audio USB speaker

ALSA device:

```
plughw:CARD=Audio,DEV=0
```

Software Dependencies

Install the required packages:

```
sudo apt update
```

```
sudo apt install \  
  build-essential \  
  cmake \  
  pkg-config \  
  libasound2-dev \  
  alsa-utils \  
  espeak-ng \  
  yt-dlp \  
  wtype \  
  wlrcctl
```

Erika also requires:

- Firefox
 - Ulauncher
 - Gemini Direct Ulauncher extension
 - Vosk
 - Vosk English speech-recognition model
-

Project Structure

```
erika/
├── Include/
│   ├── actions.hpp
│   ├── audio_capture.hpp
│   ├── command_handler.hpp
│   ├── feedback.hpp
│   ├── main.hpp
│   ├── transcriber.hpp
│   └── wake_word.hpp
├── src/
│   ├── actions.cpp
│   ├── audio_capture.cpp
│   ├── command_handler.cpp
│   ├── feedback.cpp
│   ├── main.cpp
│   ├── transcriber.cpp
│   └── wake_word.cpp
├── vosk/
│   ├── vosk-linux-aarch64-0.3.45/
│   │   ├── libvosk.so
│   │   └── vosk_api.h
│   └── vosk-model-small-en-us-0.15/
├── whisper.cpp/
├── CMakeLists.txt
├── Makefile
└── README.md
```

Architecture

Erika is divided into several components.

AudioCapture

AudioCapture interfaces directly with ALSA and continuously reads microphone samples.

The DJI microphone is opened using:

```
plughw:CARD=Rx,DEV=0
```

Audio is captured at:

```
16000 Hz
16-bit signed PCM
Mono
```

Small audio chunks are used to reduce wake-word detection latency.

WakeWordDetector

wakeWordDetector uses Vosk with a restricted grammar to detect:

```
hey jamal
```

Both Vosk partial and final recognition results are checked so the wake phrase can be detected before the speaker finishes a long silence period.

After wake detection, a short guard interval prevents the end of the word `jama l` from leaking into the command transcription.

Transcriber

Transcriber uses a general Vosk recognizer to convert the spoken command into text.

For example:

```
hey jamal  
open firefox
```

becomes:

```
open firefox
```

The recognized string is passed to `CommandHandler`.

CommandHandler

`CommandHandler` converts transcribed text into a predefined command type.

This prevents arbitrary transcribed text from being directly executed as shell commands.

Unknown commands are sent to Gemini.

Actions

`Actions` executes commands.

Actions run on a worker thread so microphone capture remains active while another command is processing.

This makes commands interruptible.

For example:

```
hey jamal  
explain what a mutex is
```

While Gemini is processing, another wake phrase can be spoken:

```
hey jamal  
stop
```

or:

```
hey jamal  
what time is it
```

The previous action is stopped and the new command takes priority.

Feedback

Feedback handles spoken responses.

Speech is generated using:

```
espeak-ng
```

and sent through ALSA to:

```
plughw:CARD=Audio,DEV=0
```

The audio path is:

```
espeak-ng
  |
  v
WAV audio
  |
  v
aplay
  |
  v
KM_B2 USB speaker
```

Spoken feedback can also be interrupted by another wake phrase.

Supported Commands

Time

```
what time is it
what is the time
tell me the time
```

Example:

```
hey jamal
what time is it
```

Erika responds verbally with:

```
The time is 7:42 PM.
```

Open Terminal

```
open terminal
open the terminal
```

Erika launches the default terminal emulator and responds:

```
Done.
```

Open Firefox

```
open firefox
open a firefox window
open a new firefox window
open firefox window
```

Erika opens a new Firefox window and responds:

```
Done.
```

Close Firefox

```
close firefox
close firefox window
close the firefox window
```

Erika focuses a Firefox window and closes one Firefox window using `Alt+F4`.

It then responds:

```
Done.
```

Play Music

Any command beginning with:

```
play
```

is treated as a YouTube search.

Example:

```
hey jamal
play bohemian rhapsody
```

Erika:

1. Uses `yt-dlp` to find the first YouTube search result.
2. Opens the result in Firefox.
3. Requests autoplay.
4. Responds with:

```
Done.
```

Firefox must allow YouTube autoplay for automatic playback to work.

Stop

Supported forms:

```
stop
cancel
stop current command
```

Example:

```
hey jamal
stop
```

This stops the current Erika action and any currently playing Erika speech.

Erika responds:

```
Stopped.
```

Gemini

Any recognized command that does not match one of the built-in commands is sent to Gemini.

Example:

```
hey jamal
explain what a mutex is
```

Erika:

1. Opens Ulauncher.
2. Types:

```
gm explain what a mutex is
```

3. Waits for the Gemini Direct extension response.
4. Reads the response from:

```
/tmp/erika_gemini_response.txt
```

5. Prints the response to the terminal.
6. Reads the response aloud.
7. Closes Ulauncher after the spoken response finishes.

Gemini Direct Integration

Erika uses the Ulauncher Gemini Direct extension.

The extension writes the generated response to:

```
/tmp/erika_gemini_response.txt
```

The Gemini Direct extension was modified so that after receiving a successful Gemini response it executes:

```
try:
    with open(
        "/tmp/erika_gemini_response.txt",
        "w",
        encoding="utf-8"
    ) as f:
        f.write(response_text)
except Exception as e:
    print(
        f"[Erika] Failed to write Gemini response: {e}"
    )
```

Erika deletes the previous response file before starting each Gemini request so stale responses are not reused.

Building

Build Erika with:

```
make build
```

or:

```
cmake -S . -B build
cmake --build build -j4
```

Running

Start normal voice-assistant mode with:

```
make run
```

Erika should print:

```
DJI microphone opened.
Listening for: hey jamal
```

It will then continuously listen for the wake phrase.

Testing Commands Without the Microphone

Commands can be tested directly from the command line.

For example:

```
make test what time is it
```



```
make test open firefox
```

```
make test close firefox
```

```
make test play erika
```

```
make test explain what a mutex is
```

List Commands

Run:

```
make list_commands
```

to display all currently supported commands.

Makefile Commands

```
make
make help
make build
make run
make clean
make list_commands
make test <command>
```

Examples:

```
make test open firefox
```

```
make test play bohemian rhapsody
```

```
make test what is the capital of japan
```

Testing the Microphone

List capture devices:

```
arecord -l
```

The DJI receiver should appear as:

```
Wireless Mic Rx
```

Test recording with:

```
arecord \
  -D plughw:CARD=Rx,DEV=0 \
  -f S16_LE \
  -r 16000 \
  -c 1 \
  test.wav
```

Press Ctrl+C to stop recording.

Testing the USB Speaker

List playback devices:

```
aplay -l
```

The current speaker appears as:

```
card 4: Audio [KM_B2 Digital Audio]
```

Test it directly:

```
espeak-ng --stdout \
  "Hello Michael, this is Erika." |
  aplay -D plughw:CARD=Audio,DEV=0
```

Using:

```
CARD=Audio
```

instead of:

```
hw:4,0
```

is preferred because USB card numbers can change after reboot.

Automatic Startup

Erika can automatically start listening after the graphical desktop session starts.

Create a systemd user service:

```
mkdir -p ~/.config/systemd/user
nano ~/.config/systemd/user/erika.service
```

Use:

```
[Unit]
Description=Erika Voice Assistant
After=graphical-session.target
```

```
[Service]
```

```
Type=simple
WorkingDirectory=/home/michaelwu/erika
ExecStart=/home/michaelwu/erika/build/erika
```

```
Restart=always
RestartSec=2
```

```
[Install]
WantedBy=graphical-session.target
```

Reload systemd:

```
systemctl --user daemon-reload
```

Enable Erika at startup:

```
systemctl --user enable erika.service
```

Start it immediately:

```
systemctl --user start erika.service
```

Check status:

```
systemctl --user status erika.service
```

View live logs:

```
journalctl --user -u erika.service -f
```

After logging into the graphical desktop, Erika will immediately launch the already-built executable and begin listening for:

```
hey jamal
```

Using the executable directly instead of:

```
make run
```

is recommended for startup because it avoids performing a CMake/build check every time the computer boots.

Whenever the source code is changed, rebuild with:

```
cd ~/erika
make build
```

and restart the service:

```
systemctl --user restart erika.service
```

Managing the Service

Stop Erika:

```
systemctl --user stop erika.service
```

Restart Erika:

```
systemctl --user restart erika.service
```

Disable automatic startup:

```
systemctl --user disable erika.service
```

Check whether it is enabled:

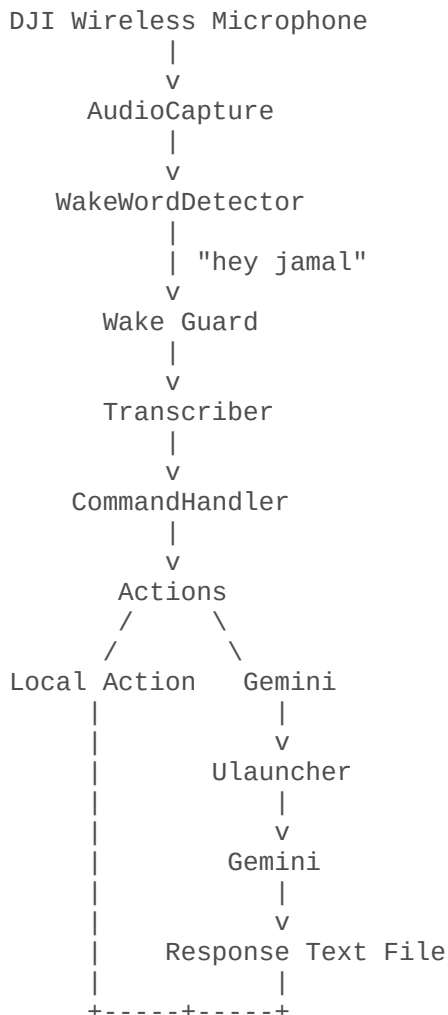
```
systemctl --user is-enabled erika.service
```

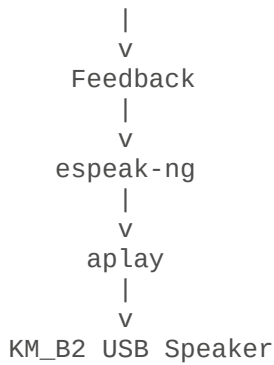
Check whether it is running:

```
systemctl --user is-active erika.service
```

Current Command Flow

The high-level command pipeline is:





Wake-Word Behavior

The assistant continuously checks Vosk partial recognition results for:

```
hey jamal
```

When detected, Erika switches to command-recording mode.

A short wake guard is used to prevent fragments such as:

```
berry  
barry
```

from the end of `jamal` being interpreted as part of the command.

The current guard interval is configured in `main.cpp`:

```
constexpr int WAKE_GUARD_MS = 400;
```

This can be adjusted based on recognition behavior.

Interrupt Behavior

The microphone continues running while actions execute.

Therefore:

```
hey jamal
```

can interrupt:

- Gemini processing
- Gemini speech playback
- YouTube searching
- Other currently running Erika actions

The new command takes priority over the previous action.

Notes

Erika currently relies on several desktop applications and external utilities:

```
Firefox  
Ulauncher  
Gemini Direct  
yt-dlp  
wtype  
wlrctl  
espeak-ng  
aplay
```

Because some actions interact with Wayland desktop applications, Erika should run as a user process inside the graphical session rather than as a root system service.

Future Improvements

Possible future additions include:

- Shared Vosk model between wake-word and command recognizers
- Whisper.cpp transcription
- Dedicated wake-word engine
- More natural TTS
- Volume controls
- Weather commands
- Calendar integration
- Timers and alarms
- Media pause/resume controls
- Window-management commands
- Hardware status commands
- More robust wake-word audio boundary detection
- Conversation context for Gemini
- Local LLM support